

Project

Anh Huynh

Task 1

You're working on software that analyzes sports players. Following are two arrays of players of different sports:

```
1  basketball_players = [  
2      {first_name: "Jill", last_name: "Huang", team: "Gators"},  
3      {first_name: "Janko", last_name: "Barton", team: "Sharks"},  
4      {first_name: "Wanda", last_name: "Vakulskas", team: "Sharks"},  
5      {first_name: "Jill", last_name: "Moloney", team: "Gators"},  
6      {first_name: "Luuk", last_name: "Watkins", team: "Gators"}  
7  ]  
8  
9  football_players = [  
10     {first_name: "Hanzla", last_name: "Radosti", team: "32ers"},  
11     {first_name: "Tina", last_name: "Watkins", team: "Barleycorns"},  
12     {first_name: "Alex", last_name: "Patel", team: "32ers"},  
13     {first_name: "Jill", last_name: "Huang", team: "Barleycorns"},  
14     {first_name: "Wanda", last_name: "Vakulskas", team: "Barleycorns"}  
15 ]
```

If you look carefully, you'll see that some players participate in more than one sport. Jill Huang and Wanda Vakulskas play both basketball *and* football.

You are to write a function that accepts two arrays of players and returns an array of the players who play in *both* sports. In this case, that would be:

```
1  ["Jill Huang", "Wanda Vakulskas"]
```

While there are players who share first names and players who share last names, we can assume there's only one person who has a particular *full* name (meaning first *and* last name).

We can use a nested-loops approach, comparing each player from one array against each player from the other array, but this would have a runtime of $O(N \cdot M)$.

Your job is to optimize the function so that it can run just $O(N+M)$.

```
1  #include <iostream>  
2  #include <string>  
3  #include <unordered_set>  
4  #include <vector>  
5  using namespace std;
```

```

6
7 // structure for each person's info
8 struct Person {
9     string first_name; // first name
10    string last_name; // last name
11    string team; // what team they're on
12 };
13
14 // function to find players who appear in both lists
15 vector<string> playsBoth(const vector<Person>& a, const vector<Person>& b){
16     unordered_set<string> fullNames; // set to store all full names from list a
17
18     for (const auto& x : a){ // for each person in first list
19         fullNames.insert(x.first_name + " " + x.last_name); // put "first + last" into set
20     }
21
22     vector<string> result; // for names that are in both lists
23
24     for (const auto& x: b){ // for each person in second list
25         string fullNames2 = x.first_name + " " + x.last_name; // "first + last"
26         if (fullNames.count(fullNames2)){ // check if this name existed in the first list
27             result.push_back(fullNames2); // if yes then add to result list
28         }
29     }
30
31     return result; // return the list for names that are in both lists
32 }
33
34
35 int main(){
36     // list of basketball players using the Person struct
37     vector<Person> basketball = {
38         {"Jill", "Huang", "Gators"},
39         {"Janko", "Barton", "Sharks"},
40         {"Wanda", "Vakulskas", "Sharks"},
41         {"Jill", "Moloney", "Gators"},
42         {"Luuk", "Watkins", "Gators"}
43     };
44
45     // list of football players
46     vector<Person> football = {
47         {"Hanzla", "Radosti", "32ers"},
48         {"Tina", "Watkins", "Barleycorns"},
49         {"Alex", "Patel", "32ers"},
50         {"Jill", "Huang", "Barleycorns"},
51         {"Wanda", "Vakulskas", "Barleycorns"}
52     };
53
54     vector<string> both = playsBoth(basketball, football); // list to store players that play
55     in both sports

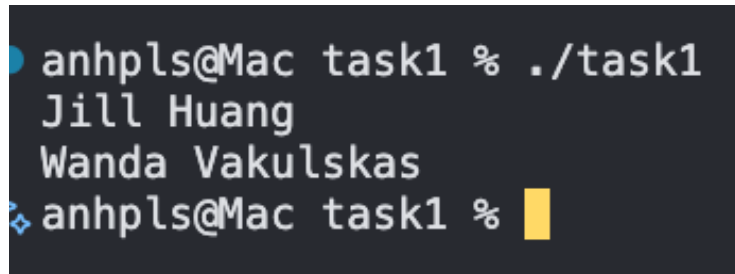
```

```

56 // print each name in both list to show which players
57 for (const string& x : both){
58     cout << x << endl;
59 }
60
61 }

```

Output:



```

anhpls@Mac task1 % ./task1
Jill Huang
Wanda Vakulskas
anhpls@Mac task1 %

```

Task 2

You're writing a function that accepts an array of distinct integers from 0, 1, 2, 3...up to N. However, the array will be missing one integer, and your function is to *return the missing one*.

For example, this array has all the integers from 0 to 6, but is missing the 4:

```
1 | [2, 3, 0, 6, 1, 5]
```

Therefore, the function should return 4.

The next example has all the integers from 0 to 9, but is missing the 1:

```
1 | [8, 2, 3, 9, 4, 7, 5, 0, 6]
```

In this case, the function should return the 1.

Using a nested-loops approach would take up to $O(N^2)$.

Your job is to optimize the code so that it has a runtime of $O(N)$.

```

1  #include <iostream>
2
3  using namespace std;
4
5
6  int returnMissingNum(const vector<int>& listOfNums){
7      int n = listOfNums.size(); // size of array 0-n
8      int totalSum = n * (n + 1) / 2; // sum of all numbers from 0 to n
9

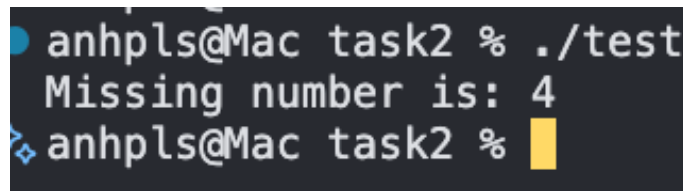
```

```

10     int testSum = 0;
11     for (int x : listOfNums){
12         testSum += x;    // sum all numbers inside the array
13     }
14
15     return totalSum - testSum; // this gives the difference between the two arrays (missing
number)
16 }
17
18
19
20 int main(){
21     vector<int> list = {2, 3, 0, 6, 1, 5}; // create populated array for testing
22     cout << "Missing number is: " << returnMissingNum(list) << endl;    // output
23
24     vector<int> list2 = {8, 2, 3, 9, 4, 7, 5, 0, 6}; // second example
25     cout << "Missing number is: " << returnMissingNum(list2) << endl; // output
26 }

```

Output:



```

anhpls@Mac task2 % ./test
Missing number is: 4
anhpls@Mac task2 %

```

Task 3

You're working on some more stock-prediction software. The function you're writing accepts an array of predicted prices for a particular stock over the course of time.

For example, this array of seven prices:

```
1 | [10, 7, 5, 8, 11, 2, 6]
```

predicts that a given stock will have these prices over the next seven days. (On Day 1, the stock will close at \$10; on Day 2, the stock will close at \$7; and so on.)

Your function should calculate the greatest profit that could be made from a single "buy" transaction followed by a single "sell" transaction.

In the previous example, the most money could be made if we bought the stock when it was worth \$5 and sold it when it was worth \$11. This yields a profit of \$6 per share.

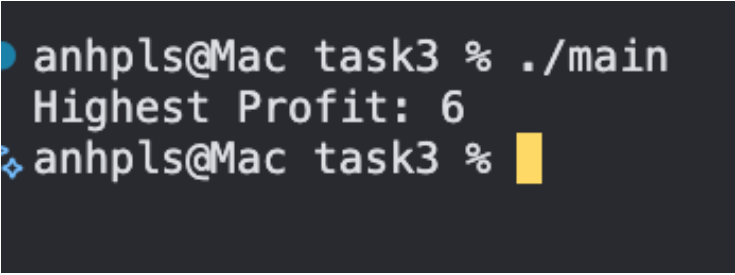
Note that we could make even more money if we buy and sell multiple times, but for now, this function focuses on the most profit that could be made from just *one* purchase followed by *one* sale.

Now, we could use nested loops to find the profit of every possible buy and sell combination. However, this would be $O(N^2)$ and too slow for our hotshot trading platform.

Your job is to optimize the code so that the function clocks in at just $O(N)$.

```
1  #include <iostream>
2  using namespace std;
3
4  int highestReturn(const vector<int>& stocks){
5      if (stocks.empty()) return 0;    // edge: handle an empty list
6
7      int minPrice = stocks[0];    // assume first day is smallest price
8      int highestProfit = 0;    // initialize to 0 profit
9
10     for(int stockPrice : stocks){    // for each stock in the stocks list
11         int currProfit = stockPrice - minPrice; // for each day, take difference between that
12         day's stock price and minimum stock price
13
14         if (currProfit > highestProfit){    // if this profit is more profitable
15             highestProfit = currProfit; // set as new highest profit
16         }
17
18         if (stockPrice < minPrice){    // if the stockPrice is lower than the smallest stock
19             price
20             minPrice = stockPrice; // set as new smallest stock price
21         }
22     }
23
24     return highestProfit;    // return highest profit
25 }
26
27 int main(){
28     vector<int> stocks = {10, 7, 5, 8, 11, 2, 6};    // test stocks list
29     cout << "Highest Profit: " << highestReturn(stocks) << endl;    // output
30 }
```

Output:



```
anhpls@Mac task3 % ./main
Highest Profit: 6
anhpls@Mac task3 %
```

Task 4

You're writing a function that accepts an array of numbers and computes the highest product of any two numbers in the array. At first glance, this is easy, as we can just find the two greatest numbers and multiply them. However, our array can contain negative numbers and look like this:

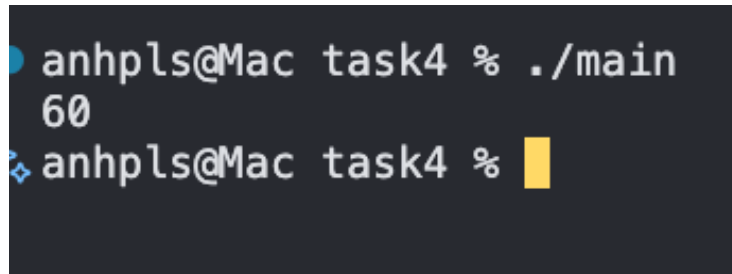
```
1 | [5, -10, -6, 9, 4]
```

We could use nested loops to multiply every possible pair of numbers, but this would take $O(N^2)$ time. **Your job is to optimize the function so that it's a speedy $O(N)$.**

```
1  #include <iostream>
2  #include <climits>
3  using namespace std;
4
5
6  int biggestProduct(const vector<int>& nums) {
7
8      int maxA = INT_MIN, maxB = INT_MIN;    // 2 largest numbers
9      int minA = INT_MAX, minB = INT_MAX;    // 2 smallest numbers
10
11     // iterate through the list
12     for (int x : nums) {
13         // update largest numbers
14         if (x > maxA) {                    // if x is bigger than the current biggest
15             maxB = maxA;                  // move old biggest to second biggest
16             maxA = x;                     // set new biggest
17         }
18         else if (x > maxB) {               // otherwise it's the second biggest
19             maxB = x;                     // set to second biggest
20         }
21
22         // update smallest numbers
23         if (x < minA) {                    // if x is smaller than the current smallest
24             minB = minA;                  // move old smallest to second smallest
25             minA = x;                     // set new smallest
26         }
27         else if (x < minB) {               // otherwise it's the second smallest
28             minB = x;                     // set to second smallest
29         }
30     }
31
32     int prodLargest = maxA * maxB;         // multiply two biggest
33     int prodSmallest = minA * minB;        // multiply two smallest
34
35     return max(prodLargest, prodSmallest); // return whichever product is bigger
36 }
37
38 int main(){
39     vector<int> list = {5, -10, -6, 9, 4}; // test list
```

```
40     cout << biggestProduct(list) << endl;    // output
41 }
```

Output:



```
anhpls@Mac task4 % ./main
60
anhpls@Mac task4 %
```

Task 5

You're creating software that analyzes the data of body temperature readings taken from hundreds of human patients. These readings are taken from healthy people and range from 97.0 degrees Fahrenheit to 99.0 degrees Fahrenheit. An important point: within this application, *the decimal point never goes beyond the tenth place*.

Here's a sample array of temperature readings:

```
1 | [98.6, 98.0, 97.1, 99.0, 98.9, 97.8, 98.5, 98.2, 98.0, 97.1]
```

You are to write a function that sorts these readings from lowest to highest.

Using a classic sorting algorithm such as Quicksort would take $O(N\log N)$. However, in this case, writing a faster sorting algorithm is possible.

Yes, that's right. Even though you've learned that the fastest sorts are $O(N\log N)$, this case is different. Why? In this case, there are limited possibilities for the readings. In such a case, we can sort these values in $O(N)$. It may be N multiplied by a constant, but that's still considered $O(N)$.

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  vector<double> sortTemp(const vector<double>& temps) {
6
7      const int RANGE = 21;                // total possible temps from 97.0 to 99.0
8      vector<vector<double>> buckets(RANGE); // 21 buckets to hold each temp group
9
10     // drop temps directly into their matching bucket
11     for (double t : temps) {
12         int index = int((t - 97.0) * 10); // temp like 97.3 becomes index 3
13         buckets[index].push_back(t);     // throw the value into that bucket
14     }
15 }
```

```

16     vector<double> sortedList;                // final sorted temps
17
18     for (int i = 0; i < RANGE; i++) {         // go bucket by bucket in order
19         for (double t : buckets[i]) {         // grab everything inside the bucket
20             sortedList.push_back(t);          // push it into the sorted list
21         }
22     }
23
24     return sortedList;                        // return the sorted temps
25 }
26
27 int main() {
28     vector<double> data = {98.6, 98.0, 97.1, 99.0, 98.9, 97.8, 98.5, 98.2, 98.0, 97.1}; //
example data
29     vector<double> sorted = sortTemp(data);    // created the sorted list
30
31     for (double x : sorted) {                  // print sorted temps
32         cout << x << " ";                    // print each temp with a space
33     }
34 }

```

Output

```

anhpls@Anhs-MacBook-Pro task5 % g++ -std=c++17 task5.cpp -o main
anhpls@Anhs-MacBook-Pro task5 % ./main
97.1 97.1 97.8 98 98 98.2 98.6 98.5 98.9 99 %

```

Task 6

You're writing a function that accepts an array of unsorted integers and returns the length of the *longest consecutive sequence* among them. The sequence is formed by integers that increase by 1. For example, in the array:

```
1 | [10, 5, 12, 3, 55, 30, 4, 11, 2]
```

the longest consecutive sequence is 2-3-4-5. These four integers form an increasing sequence because each integer is one greater than the previous one. While there's also a sequence of 10-11-12, it's only a sequence of three integers. In this case, the function should return 4, since that's the length of the *longest* consecutive sequence that can be formed from this array.

One more example:

```
1 | [19, 13, 15, 12, 18, 14, 17, 11]
```

This array's longest sequence is 11-12-13-14-15, so the function would return 5.

Your job is to optimize the function so that it takes $O(N)$ time.

```
1  #include <iostream>
2  #include <vector>
3  #include <unordered_set>
4  using namespace std;
5
6  // function to find the largest consecutive sequence
7  int consecutive(vector<int>& list){
8      unordered_set<int> a;    // init a hash set to store all nums
9
10     for (int x : list){      // loop through the vector
11         a.insert(x);         // insert each number into the set
12     }
13
14     int longest = 0;         // init the longest sequence to 0
15
16     for (int x : list){      // check each number in the list
17         if (a.find(x-1) == a.end()){ // only start counting if x-1 is not in the set
18             int curr = x;    // current number in the sequence
19             int length = 1;  // length of the current sequence
20
21             while (a.find(curr + 1) != a.end()){ // while the next number exists
22                 curr++;      // move to the next consecutive number
23                 length++;    // increase the sequence length
24             }
25             longest = max(longest, length); // update the longest if this sequence is bigger
26         }
27     }
28     return longest; // return the longest sequence length
29 }
30
31 int main(){
32     vector<int> list = {10, 5, 12, 3, 55, 30, 4, 11, 2}; // first test list
33     cout << consecutive(list) << endl; // should print 4
34
35     vector<int> list2 = {19, 13, 15, 12, 18, 14, 17, 11}; // second test list
36     cout << consecutive(list2) << endl; // should print 5
37 }
```

Output

```
anhpls@Anhs-MacBook-Pro task6 % g++ -std=c++17 task6.cpp -o main
anhpls@Anhs-MacBook-Pro task6 % ./main
4
5
```

Video Link:

<https://youtu.be/dYYB-7yYxIM>